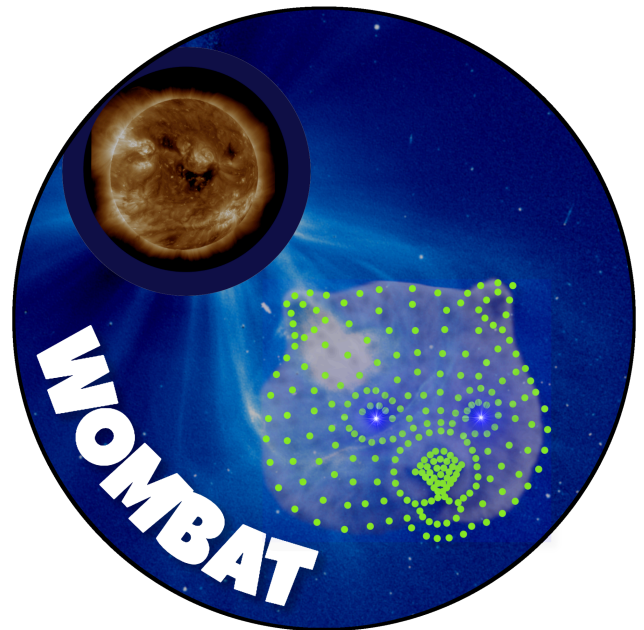


WOMBAT (+ DINGO)

This is a basic guide on how to use the Wireframe Object Mapped onto Background Alignment Tool (WOMBAT) and the Density Inferred from a Nice Grid Object (DINGO) companion. We recommend using the “pull” and “process” routines packaged with WOMBAT to download and prep observations to use in the GUI. The pull function is optional but the process is necessary to pre-process things to make the GUI run efficiently. The GUI has most thoroughly been tested with observations from pull/process. If you do not care about details there is a cheat sheet on the last page.

Table of Contents

1. [Installing Wombat and Basic Procedure](#)
 - 1.1. [Initial Setup](#)
 - 1.2. [How to WOMBAT](#)
2. [Acquiring Data](#)
 - 2.1. [Running the Script](#)
 - 2.2. [Instrument Tags](#)
 - 2.3. [File Structure](#)
 - 2.4. [General Tips](#)
3. [Processing the Data](#)
 - 3.1. [Running the Script](#)
 - 3.2. [Prep Details](#)
 - 3.3. [Prep Output](#)
4. [Launching the WOMBAT GUI](#)
 - 4.1. [Using a Pickle](#)
 - 4.2. [Using a Log File](#)
5. [Using the WOMBAT GUI](#)
 - 5.1. [Parameter Window](#)
 - 5.2. [Observation Window](#)
 - 5.3. [Overview Window](#)
 - 5.4. [Mass Calculation](#)
 - 5.5. [Working with WOMBAT](#)
 - 5.6. Keyboard Shortcuts
6. [WOMBAT Post-Processing](#)
7. [DINGO](#)
 - 7.1. [Running DINGO](#)
 - 7.2. [DINGO Results](#)
8. [Example Cases](#)
9. [Appendix](#)
10. [WOMBAT Cheat Sheet](#)



1. Installing WOMBAT and Basic Procedure

1.1 Initial Setup

WOMBAT is simple enough to not really need any sort of specialized virtual environment. Most of the packages used are straightforward and likely already installed if you have been doing python-based solar research. A few packages related to the GUI may need to be installed. The Appendix (section A1) lists the required packages and their version number. During WOMBAT development any new needed packages were installed via pip and the same should be sufficient for an initial setup. We have also provided a yml file if one wishes to set up a WOMBAT environment for conda. See the note in A1 about ffmpeg if you have trouble making movies.

Download the WOMBAT repository from the [Github site](#). The wombat directory will have six top level python scripts, two folders, and a few other text files. The python files correspond to the WOMBAT pull, process, and GUI script and three post-processing scripts (DINGO plots, WOMBAT plots and movies). The prepCode folder contains the code used by the processing scripts and the wombatCode folder contains scripts related to the GUI itself.

Download the support files from the [Zenodo repository](#). This contains two zip files, one for SPICE kernels and one for various files needed for the processing scripts. We give more details on the contents of each in the Appendix in A2 and A3. Unzip each in the main wombat directory (i.e. wombat/spiceKernels/ and wombat/prepFiles/).

There is no need to execute any install command. As long as all the downloaded components are placed together and the correct python libraries exist it should be ready to run.

1.2 How to WOMBAT

We provide a pull and process script to set up the background observations for use with WOMBAT. The pull script is optional, but the process script is required and has been most thoroughly tested using the fits versions obtained through the pull script. These scripts are set up to be called from the command line, but this is enabled through wrapper functions so one could call their core functions from an external script if desired.

The pull script only needs to be run once for a set of observations. The process script performs basic background calibration, creates running and base difference images, scales them to nice ranges for the GUI, and calculates mass images. All this information is saved in a python pickle (binary save file). This allows the GUI to quickly load as we have already done all the heavy processing. The pickle can be repeatedly used to run the GUI without rerunning the process script, however if one wants to change the time ranges or the instruments included it will need to be rerun.

The step to pull data is

```
python wombatPullObs.py time1 time2 instruments optionalParameters
```

and followed by the processing step

```
python wombatProcessObs.py time1 time2 instruments optionalParameters
```

where the times, instruments, and optional parameters will be explained in further detail in Sections 2 and 3. Afterwards the GUI can be run on these observations via

```
python wombatWrapper.py wbPickles/WBpickle.pkl optionalParameters
```

where the WBpickle.pkl is the pickle from the processing script and the optional parameters are further explained in Section 4. Python 3 should be used so your call to python may be “python3” depending on your setup.

2. Acquiring Data

The WOMBAT pull script is essentially a wrapper to call the Fido procedure from Sunpy, which downloads observations from remote repositories. The pull script functions as well as Fido functions, which is to say sometimes. Connecting remotely seems to be an issue easily affected by VPNs, firewalls, the weather, quantum physics, and other random undiagnosed issues. The network on which WOMBAT was developed does not play nice with downloading certain satellites via Fido, but we have a work around to pull the filenames then directly download from a different source using wget. Fixing Fido issues is beyond the scope of the WOMBAT project. The pull script will work for each data source ***if the connections are happy***. If it fails or hangs up sometimes just killing and restarting the process will work. Alternatively it may require trying again at a later time or resorting to manual downloads.

2.1 Running the Script

The basic call is

```
python wombatPullObs.py time1 time2 instruments optionalParameters
```

where time1/time2 set the time range over which to pull observations and instruments is one or more instrument keywords from the list of available options below. The times can be in any format readable by parse_time from Sunpy, we recommend using YYYY-MM-DDTHH:MM (or / instead of -). Both times and at least one instrument keyword are required and must be given in order as “time1 time2 instruments” before any optional parameters.

The first type of optional keyword is a time resolution to pull. Fido will automatically grab every available matching observation within the time range, which may be more than ideal to load into a GUI. We group observations by type, either EUV, COR (coronagraph), or HI (heliospheric imager). A time resolution is set by passing the type plus “T” and a number (e.g. EUVT10 for 10 min EUV observations). The wrapper is not sensitive to case and will handle multiple time resolutions being passed. It will default to 10 min resolution for EUV, 20 for COR, and 30 for HI.

The other optional parameter is a directory where the downloaded files should be saved. The wrapper will assume that any optional parameter that doesn't have a time resolution flag in it (e.g. CORT) is an output folder. ***That folder must exist before passing it*** as an optional parameter. The default location is wombat/pullFolder/

2.2 Instrument Tags

Only allowed instrument tags can be passed to wombatPullObs. The available options are:

Satellite	Instrument Tags
PSP	WISPR, WISRPI, WISPRO
SDO	AIA94, AIA131, AIA171*, AIA193*, AIA211, AIA304*, AIA335, AIA1600, AIA1700 *most frequently used/best-tested
SOHO LASCO	C2, C3
Solo	SoloHI, SoloHI1, SoloHI2, SoloHI3, SoloHI4, EU1174, EU1304
STEREO	EU1171, EU1195, EU1284, EU1304, COR1, COR2, HI1, HI2 *Can add A or B on the end

Most of the tags should be self explanatory as shorthand for the instrument names. For the EUV instruments the number represents the wavelength. For both PSP/Solo, giving the generic name (WISPR/SoloHI) will pull both inner and outer or all four quadrants, but one can use the other tags to be more specific.

2.3 File Structure

The downloaded files are saved within subfolders within pullFolder/ (or the user-specified folder). Each satellite has a folder, then instrument folders within, then possibly wavelength or camera folders. For example:

pullFolder/PSP/WISPR/L3_Inner/
or
pullFolder/STEREO/EUVIA/171/

The wrapper is set up to automatically generate this nested folder structure and the process folder will look for files stored as such, even if the top folder is named something other than pullFolder.

2.4 General Tips

In theory, one can pass a full list of instrument tags and it should pull all of them at once. It seems like Fido is less likely to get overloaded if one passes only single instruments or a few at a time. We recommend splitting things up into smaller sets (e.g C2/C3 separate from COR1/COR2).

The AIA downloads are the slowest, which is reasonable given the file sizes, but it also seems to be a relatively unstable connection to the data source. At one point during development, AIA images (that we had previously been able to download) became inaccessible for no apparent reason but began working fine a week later with no changes to the code. Other times it works but takes of order a minute per file.

Fido queries the available observations for PSP and SolO without issue, but the downloads consistently fail for the lead developer (presumably due to network/firewall issues of their institute). As a work around, we can take the filenames from the Fido query and use wget to pull them from the NRL instrument sites (see online at [PSP](#) and [SolO](#)) since they use the same file structure and naming system. We have not tested this outside the US and acknowledge that this work around may have issues internationally.

In Appendix A4 we also provide links to external sites where the data can be found and downloaded by hand (or writing your own wget commands). For LASCO data, the renameLASCO script (hiding in prepCode) must be run to convert the sequential numbered fits into names with dates so the process script can interpret them.

3. Processing the Data

3.1 Running the Script

The basic call is almost the same as the pull script, but using the process file

```
python wombatProcessObs.py time1 time2 instruments optionalParameters
```

where time1/time2 set the time range over which to pull observations and instruments is one or more instrument keywords from the list of available options. The times can be in any format readable by parse_time from Sunpy, we recommend using YYYY-MM-DDTHH:MM (or / instead of -). Both times and at least one instrument keyword are required and must be given in order as “time1 time2 instruments” before any optional parameters.

The process script has three optional parameters. The first is the folder where the unprocessed data is stored. This defaults to pullFolder (the same as the default output for the pull script) but can be something else as long as the structure matches with the same satellite/instrument/wavelength or camera subfolders. The script will automatically process whatever data is in this folder that matches the time range and instrument tags. The next parameter is a save name for the.pkl file. If a string including ‘.pkl’ is passed this will be set as

the save name for the output file, otherwise WBGUI_temp.pkl will be used. The final parameter is an option to turn off the mass calculation, which saves some computation time and shrinks the size of the pickle. This is flagged by adding “nomass” in the command line. Note that DINGO cannot use a massless pickle.

3.2 Prep Details

As mentioned above, the prep code not only applies the routine processing techniques to the various observations, but it performs all of the intensive calculations that can be done outside the GUI to maximize the efficiency when using the tool. First basic processing is done to convert images to calibrated total brightness. Then, for the white light images, a base difference is taken and the excess total brightness is converted to an integrated mass per pixel. We also determine a running difference for each instrument. The set of base and running differences are then mapped onto “nice” ranges to display in the GUI for three different display modes - linear, logarithmic, and square root. The process script can take a few minutes for a day+ long time span with multiple instruments, but only needs to be repeated if you want to change the instruments included or the time range.

The prep wrapper uses a set of processing scripts for different satellites/instruments, all stored in prepCode. The majority of these are stripped down ports of the IDL SolarSoft (SSW) prep scripts. Processing the files often requires background images, SPICE kernels, and other satellite/instrument parameters, most of which we have included via the zip files from Zenodo.

The one exception is for STEREO COR1, ***which requires a trio of images specific to the year and month of the observations that allow us to convert the polarized triplet into a single image.*** These support files are available via NASA [here](#). Pick the corresponding satellite (A or B), then monthly_min, then the YYYYMM corresponding to the year and month. Create a folder STEREObackgrounds/ in the main wombat directory. In here we will mirror the structure of the NASA site so create folders a/ and b/, each with a monthly_min/ inside. Then create a folder for the year+month in a YYYYMM format. Download the 9 mc1A_p### or mc1B_p### files and place them in their year+month folder. Technically, only one of each p### is needed, whichever is closest to the actual date, but downloading all is typically quicker than doing the math or sorting out borderline cases. Right now we only support the polarized triplets for COR1, not COR2, but future updates would require similar files for COR2 triplets.

The processing scripts will work with the specific file versions downloaded by the pull script. They should work with the same versions acquired elsewhere, but different versions may or may not be supported. The following table lists the intended input version/level of each instrument, the corresponding processing script, and any additional notes.

Inst Tag	Type/Level	Prep Code	Notes
AIA#	Level 1 aia.lev1.WAV...fits	aia_prep	Uses aiapy, not a SSW port. Updates pointing, registers, and normalizes (no respike/deconvolve)
C2/C3	Level 0.5 Source is img#.fits Pull adds date string	lasco_prep c2_prep c3_prep	SSW port. Both require files in prepFiles for exp_factor and C3 uses additional fits files there
COR1	Tags s4cX, s5xX, n4cX (X is A/B)	secchi_prep cor_prep	SSW port. Requires download of appropriate monthly_min backgrounds. Will take list of files and sort into the correct pB triplets
COR2	Tags d4c2X (X is A/B)	secchi_prep cor_prep	SSW port. Works with total brightness or pB triplets
EUI#	Level 2 solo_L2_eui-fsi...Vo2	(No external code)	Not a port. Just uses sunpy map and ImageNormalize
EUVI#	Tags n4euX, n5euX (X is A/B)	secchi_prep eui_prep	SSW port. Depends on files in prepFiles
HI1/HI2	Tags s4h1X, s4h2X (X is A/B)	secchi_prep hi_prep	SSW port. Depends on files in prepFiles. Can add tag '_SR' on end to run star removal routine in processing.
SoloHI, SoloHI1- SoloHI4	Level 2 Tags 1ft, 2ft, 3ft/g, 4ft/g	solohi_prep	SSW port. No real image processing needed for L2 but combines into mosaic if not passed a single instrument tag
WISPRI, WISPRO	Level 3 psp_L3_...1221.fits psp_L3_...2222.fits	wispr_prep	SSW port. Depends on files in prepFiles. Can add tag '_LW' or '_L2' to use either LW or L2 files instead (must pull and place in pullFolder manually)

We note that the secchi_prep ports are near identical matches to the IDL routines. We made a line by line port to maintain a pixel-by-pixel match so that any quantified results (total brightness/derived CME mass) are directly comparable to previous IDL results. The results typically differ by <0.1%, with the small variation introduced by minor differences in IDL/Python interpolation routines.

We currently support other tags for HI and WISPR. For HI, adding '_SR' on the end of a tag will tell the process code to run the star removal (SREM) program and replace the standard running difference with these results. *****SR files also need backgrounds similar to COR1***** For WISPRI, [L2](#) or [LW](#) processed data can be used in place of L3. The LW will replace the difference images but use the calculation from L2 images to get the mass. Multiple tags for

the same instrument (e.g. WISPRI and WISPRI_LW) will work and create a panel for each in WOMBAT.

3.3 Prep Output

The fits files are processed and the results are saved in a pickle in the folder `wbPickles`, which the script will create if it does not already exist. It defaults to saving it as `WBGUI_temp.pkl` and will overwrite an existing file of the same name. It is recommended to either rename the temp file or pass a unique name as an argument if you would like to be able to restore this background set up without waiting for the processing in the future.

4. Launching the WOMBAT GUI

Once the observations are processed we can launch the GUI. The only provided way to launch the GUI is using a pickle from the prep script. The GUI code is modular, however, so one could write a different wrapper script suited to their own purposes so long as it feeds the appropriate inputs to the GUI.

4.1 Using a Pickle

The most basic launch method just passes the results of the processing script to the GUI wrapper along with a few additional parameters.

```
python wombatWrapper.py wbPickles/WBGUI_temp.pkl nWFs ovw zoom
```

Here the first argument is the name/path to the pickle, the second is the number of wireframes to include, and the third is a flag to turn on the overview window. We have set an upper limit of 5 wireframes, which should be sufficient for any reasonable use case. The overview window (ovw, discussed in 5.3) is not automatically included if the tag is not provided. One can include the tag `zoom` to allow for panning and zooming within the observations. The aspect ratio is kept constant but it can be difficult to return to the original view so this defaults to off for any white light images but is allowed without the tag for EUV images.

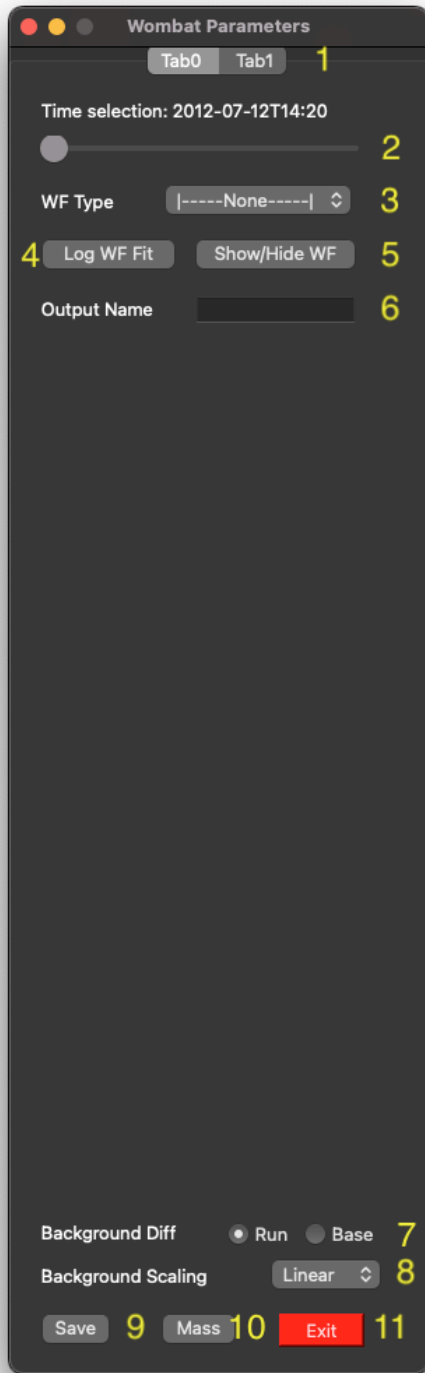
This method of launching WOMBAT is used the first time you are working with a new set of observations. If you have logged results from a previous session then you can reload them using the following options.

4.2 Using a Log File

The alternate option is to pass a log file and the lines of interest to the wrapper

```
python wombatWrapper.py wbOutputs/WomBlog.txt indices ovw zoom
```

where the first argument is the log file and the second is the selected lines within this file. The `ovw` tag can still be used when loading from a log file. The log file is a file generated by



WOMBAT when one “logs” fits within the GUI and WomBlog.txt is the default save name. There are several options for formatting the index string. It can be a single integer (e.g. 1), a range of integers (e.g. 1-5), multiple integers separated by + (e.g. 3+5+7), or a combination of those formats (e.g. 1-5+7+9-11 where the ranges will be interpreted first). The log file includes all the wireframe parameters, the time of the reconstruction, the background pickle used in the GUI, and the scaling mode, difference mode, and contour levels of the background. The maximum of 5 wireframes still applies and all lines must use the same background pickle, but you can pull in multiple time steps for each wireframe. All observation times stored within the pickle will be loaded but it will only use the reconstructions at the given times. Times before/after the selected reconstructions will be set at the earliest/latest provided values.

5. Using the WOMBAT GUI

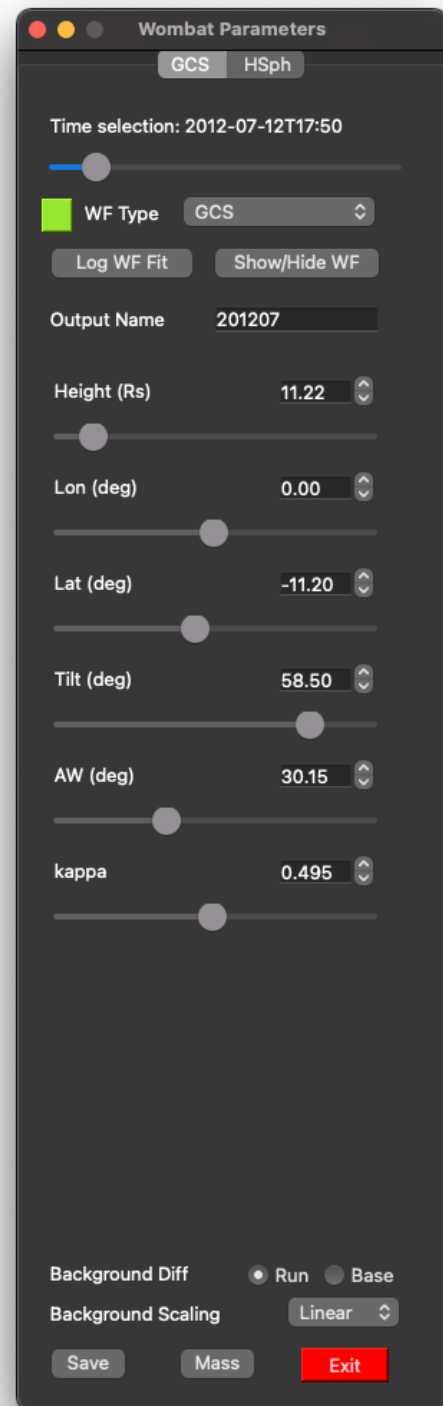
Once the background files have been prepped and the GUI has been launched the use should be intuitive (hopefully). The GUI will open one parameter window and one observation window for each instrument. The overview window is also launched if the ovw keyword is passed to the wrapper script. The following shows the GUI as launched from a background file with everything unset or set at default values. If a log file is used then various parameters will be initialized at the earliest specified configuration.

5.1 Parameter Window

The figure to the left shows an example of the parameter window on start up. We have added the yellow numbers so we can highlight each of the available features.

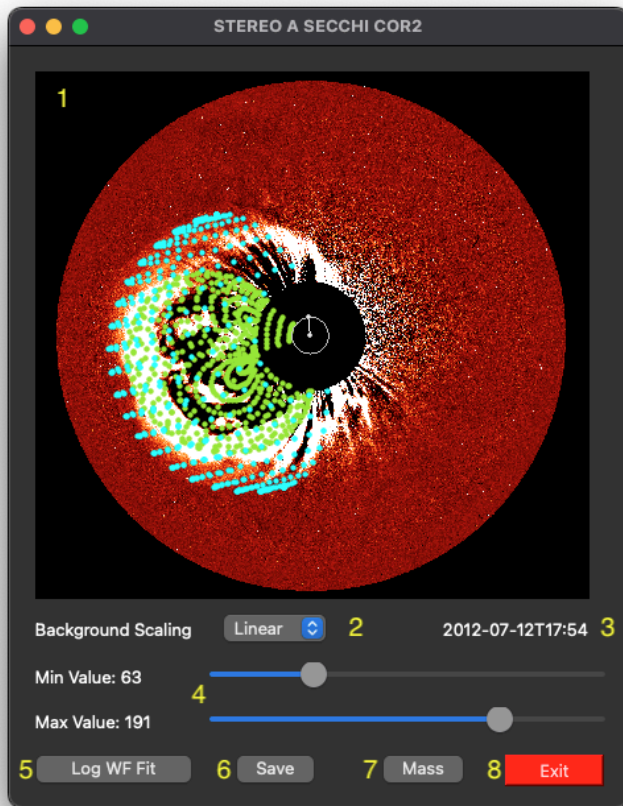
1. Wireframe (WF) tabs. For this example, we have initiated the GUI with two WFs so the top row has two tabs with generic tab names. One can toggle through the WFs by clicking on the tabs.
2. Time slider bar. This is a universal time across the GUI and the observation windows will show the observation closest to that time for each instrument.

3. WF Color selector. If the box is clicked a color selection tool will pop up and can be used to set the color of the current wireframe. The default is canonical GCS neon green. It will automatically change to the default for each WF type once the type is set.
4. WF Type drop down box. This is where one selects the WF type for this tab. There are currently nine different WF options. Once a WF is selected the large gap in the middle of this window will populate with parameter sliders. Setting the WF type back to none will clear the parameter sliders and the WF from the observation windows. Note that GCS* is GCS with the option to move the origin off the Sun-center.
5. Log WF Fit button. This will record the current values for all WFs at this time step in the log file and print them to the terminal. These log lines can be used later to initiate the same WFs
6. Show/hide WF button. This toggles whether or not a WF is viewable in the observation windows. It keeps all the parameters/values at the same settings, unlike setting the type to none, but hides the WF scatter points.
7. Output name text box. An option to change the save names from the default values for the log file. This does not affect figure names.
8. Background difference radio button - Option to flip between a running difference or base difference for the background images. If an alternate fancy version of the image is passed to the wrapper (i.e. the star removal HI images) then this will replace the running difference image.
9. Background scaling drop down box. This selects between different methods of scaling the observations for all background windows. Option to choose between linear, logarithmic, or square root scaling.
10. Save button to save a png figure for each observation window and the overview window.
11. Mass button to calculate the total mass enclosed by the projection of a WF. The first press will show a mask of the selected region and print values to the terminal. The second press clears the mask. This will work with multiple WFs but each WF is calculated individually as if the others do not exist so the value will not be very accurate.



12. Exit button to close everything.

Once a WF type is from the drop down box the middle part of the GUI fills in with the widgets to set up the corresponding parameters. Each parameter will have a label, a text entry box, and a slider. Note that enter/return must be pressed for the contents of a text entry box to be read in. All parameters have a height, longitude, and latitude but the additional parameters defining the shape and orientation vary between WF types. Each parameter will be automatically set to a “nice” starting value and the WF will appear in the observation window. The panel to the left shows an example for when the GCS WF type is selected.



The height slider range will adjust itself based on the range of the observations. In the example, we have loaded a pickle with STEREO HI1 so the height runs from 0-120 Rs. If we only had EUV and/or COR type observations range would be smaller and the slider would have fine resolution. In general, the text box and up/down arrows on the spin box can be used to set the parameters more precisely than the sliders. We note that the height is the front distance for all WF types except for the slab where it represents the center.

5.2 Observation Window

To the left, we show an observation window with two WFs fit to the observations. This panel shows STEREO A COR2 images, but we will have one window for each instrument we have selected. Within the figure we have a green GCS shape fit to the main ejecta

and a blue half sphere fit to the CME-driven shock. Again, yellow labels have been added for illustrative purposes within the guide.

1. Display window. This is where we show the observations and the wireframe projections. Note for EUV observations we allow the built in zoom and pan operations since one may want to focus in on a small region/limb but these have been disabled for coronagraph and heliospheric imager observations. The window panel will scale as the full window is resized but will maintain a constant aspect ratio.

2. Background scaling drop down box. This selects between different methods of scaling the observations. Option to choose between linear, logarithmic, or square root scaling. This is the same as the parameter window version, but will modify only this window.

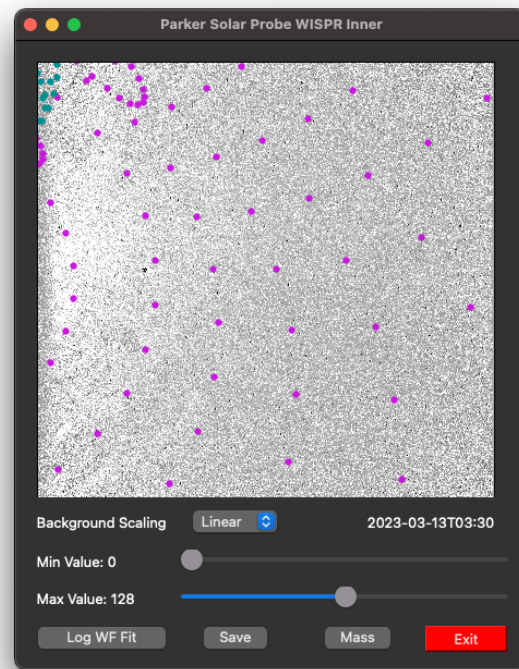
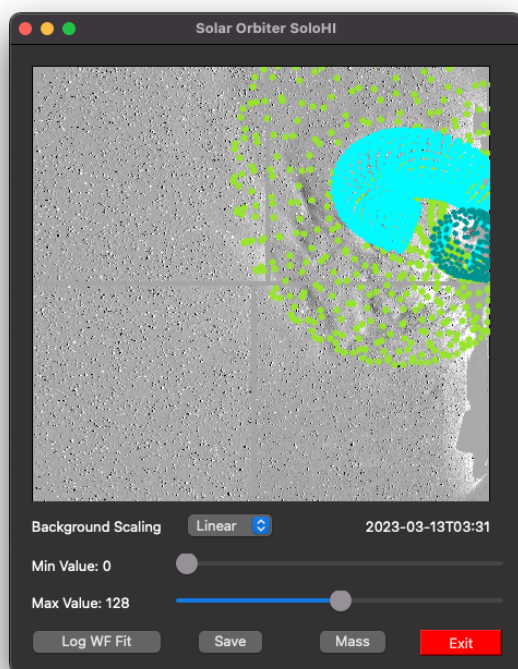
3. Time stamp. This is the actual time of the observation, which should be close to but not an exact match to the global slider time.
4. Min/max value sliders. Options to set the visible range of the background. Decreasing the minimum will make dim features brighter and decreasing the maximum will make bright features brighter. The GUI attempts to initiate all types of observations at nice values for all types of scaling, but adjustments may be necessary to highlight different features in an image.
5. Log WF Fit button. This is the same as the parameter window version but will only log the current parameter values for the observations in the selected window
6. Save button to save this panel as a png
7. Mass button to determine the mass enclosed by the projection of the WFs. This is the same calculation as the main window but will only process the current window
8. Exit button to close everything.

We have attempted to make the windows automatically show nice ranges for every observation, but this will likely not always work. If you need to adjust the dynamic range of the visualization (beyond what the sliders can do) you can change the values in the processing script. Search for `#tagIt:dynrng` and then rerun it.

5.3 Overview Window

The next two figures show examples using SoloHI and WISPR as background images. This is the 2023 March case from the WOMBAT paper, here we show only three of the wireframes to illustrate a few more WOMBAT features.

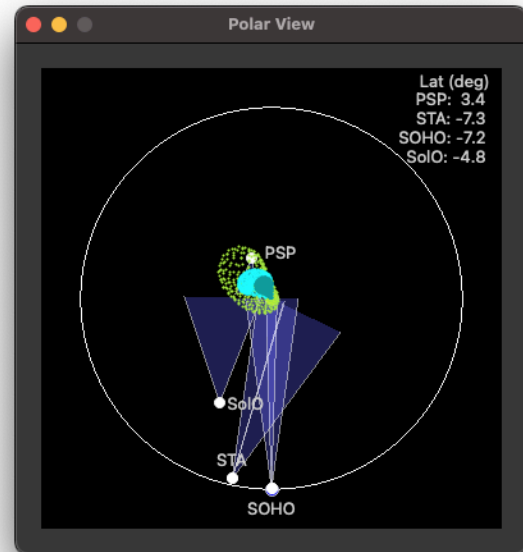
To better understand what is going on in these observation panels we can look at the corresponding overview window, shown below. This window is just a visualization, there are no interactive features. The panel shows a top down view on the ecliptic plane. The Sun and Earth are shown as yellow (buried under wireframes here) and blue circles. The Earth's orbit is shown



as a white circle. Any satellites being used in the GUI are shown as white dots labeled by the satellite name and their latitudes are shown in the upper right corner. This example includes all the satellites present in the pickle, not just the two shown above. The field of view (FoV) of each satellite is shown with a blue wedge (visible for SolO/STA/SOHO but obscured for PSP in the image). The FoV length is set by the distance to the Thomson surface at the center of the FoV. The overview also includes the projection of each WF on the ecliptic plane.

For this configuration we show three GCS wireframes on the far side of the Sun, the colors in the overview match the colors used for the projections onto the observations. The farthest of these wireframes (lime green) overlaps with PSP. WOMBAT does a very cursory check and if a satellite is within a WF then any points that happen to be visible are switched to purple. The example shows a relatively well-behaved set of purple points but weird projections can occur leading to an odd distribution of points. The switch in color is only meant to flag a potential overlap, it should not be taken as fact without a more accurate calculation.

A similar switch in color is used for EUV images (not shown). When a WF is well outside of the FoV the observation panel will show the projection of the WF onto the solar surface in the same purple color.



5.4 Mass Calculation

The mass button will call a routine that determines an outer bound of each wireframe using a convex hull. After pressing the button, each observation panel will show the mask with regions outside it darkened. WOMBAT then uses this mask and sums up the corresponding values from the pre-calculated mass images. The calculated values will be printed to the terminal. Pressing the mass button a second time will clear the mask from the observation panels. If you are using a pickle created using the nomass tag then it will print nothing.

WOMBAT does calculate masses when there are multiple WFs but it treats each one individually. To do this it completely ignores any overlap so a pixel included in multiple WFs will add its mass multiple times. As such, we do not recommend blindly using the mass button in these cases. At a minimum one should at least approximate a sheath mass by subtracting an inner CME mass from an outer sphere mass. Alternatively, the DINGO code has a better option to separate the masses between these overlapping regions during post-processing using the logged results. In general, the DINGO calculation is recommended over the WOMBAT values but the quick in-GUI values are fine for isolated events.

The normal calculation yields projected masses so they will likely not agree for the same wireframe in different instruments. The GUI will also estimate a deprojected mass using the longitude of the wireframe relative to the PoS for that instrument. These values should be closer between instruments, but there are many oversimplifications introduced and there may be legitimate differences due to differences in the occulters or perspectives obscuring part of an event. For more information on the mass calculations see [Kay et al. \(2025\)](#).

5.5 Working with WOMBAT

WOMBAT can be used for single image/time fits but has been designed for tracking features over time through multiple FoVs. To facilitate this, it retains a memory of the settings last used for every combination of time and wireframe or observation panel. The WFs are tracked separately for each tab if using more than one WF.

The memory is initially set to default values unless reloading from a log file. When using logged results, times before/after the provided range will be set to the earliest/latest provided values. If a WF type has not been used before, either from a fresh start or simply not provided in the log results, then it will load at default values. Once these are adjusted and the time is changed the first set of custom values will be propagated to all times. After this initialization, all changes will only affect and be stored for the current time. If switching to a new WF type from an existing type, WOMBAT will try and match parameters as makes sense (e.g. keeping the same latitude and longitude) and set the remainder at default values.

The observations and global GUI time (as given by the slider) are not exactly matched and the global time usually has higher resolution. The state memory uses the global time indexing, and simply determines which global indices are most closely matched to the current observation time. The instrument settings are kept individually for each observation window so there is no complication in tracking their changes. For the WF parameters, however, there is a single set of WF parameters using the global time, which must be mapped to all the instruments with their differing time cadences. See Appendix 5 for an unnecessarily complicated description of this but essentially the GUI will remember the settings for the smallest chunk of observational time matching the slider time.

We include shortcut keys that allow one to propagate the WF and observation settings back and forth in time, rather than being isolated to the current time. See the list below for specifics. We recommend first getting a fit to a time where the feature can be fit well (probably not the earliest or latest time) then propagating the parameters forward and backward over all times. From there, work forward adjusting the parameters as needed for the next timestep, then propagating the new values forward before moving the next time step. The same process can be repeated for the times earlier than the initial fit, propagating the revised fit backwards before moving to earlier times. Whether one works forward or backward first is irrelevant. Note that there are different shortcuts for propagating all WF parameters versus just the WF in the current tab (if using more than one WF).

Once the reconstructions have been made one can step back and forth in time and see the evolution of the WF structure. The WF can be adjusted and it will only affect the current time step, allowing for fine scale tuning of the results. Once the reconstructions are finished one can step through time and log the results. If the main parameter window is highlighted when the shortcut is hit it will log the results for all wireframes for all instruments, including the observation aesthetic settings. If a single observation window is highlighted then it will only record that instrument. There is a shortcut to log everything (all WFs for all windows for all times). We recommend doing this manually due to the difference in time cadences and ***making sure the global time is the closest match to each individual observation time.*** If one instrument has a lower cadence than another you may have adjusted the global wireframe parameters to have multiple values corresponding to the same observation time for the low cadence instrument.

WOMBAT does have one additional feature where if one clicks within the display portion of an observation window it will print the position of the cursor to the terminal. It will give the pixel location, the corresponding SkyCoordinates (Tx/Ty) from astropy, and the project radial distance and position angle. If the pickle has masses for this instrument then it will print the mass within that pixel.

5.6 Keyboard Shortcuts

WOMBAT includes many shortcuts that make use of the keyboard. Some of these perform the same actions as buttons in the GUI and are simply for convenience. Others allow access to features that one cannot reach by clicking on windows. Note that if the focus is currently in a text entry box (output name or parameter value) the keyboard shortcuts will not register. Simply press enter to escape the box and the shortcut should now work.

The first set of shortcuts are basic controls that work the same regardless of what window is highlighted when they are pressed.

- **Q** - Close the active window
- **Esc** - Close all windows
- **H** - hide/show wireframes
- **Right** - step forward along the time slider
- **Left** - step backward along the time slider

The following set of shortcuts have both a normal option and a Shift+ action (i.e pressing shift plus that key) that will perform the same behavior but for all times instead of just the current time. All of these function on a single observation window if that is the active window and will apply to all the observation windows if the parameter window is active.

- **L** - Add the current parameters to the log file
- **S** - Save the current image.
- **R** - Select current time to running diff
- **B** - Select current time to base diff
- **1** - Select current time to linear scaling
- **2** - Select current time to logarithmic scaling

- **3** - Select current time to SQRT scaling
- **M** - Determine mass within each WF at this time

The final set of shortcuts have only Shift+ actions. These are actions that affect large portions of the global state memory so we do not want them triggered on a single accidental key press.

- **Shift+7** - Propagate this WF parameters back in time (technically “&”)
- **Shift+8** - Propagate this WF parameters forward in time (“*”)
- **Shift+9** - Propagate current obs min/max slider values back in time (“(”)
- **Shift+0** - Propagate current obs min/max slider values forward in time (zero not O, “)”)

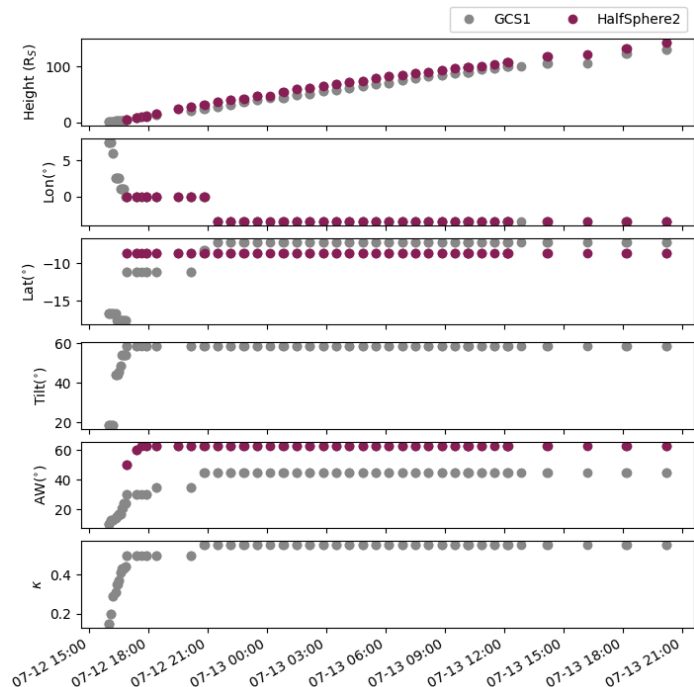
6. WOMBAT Post-Processing

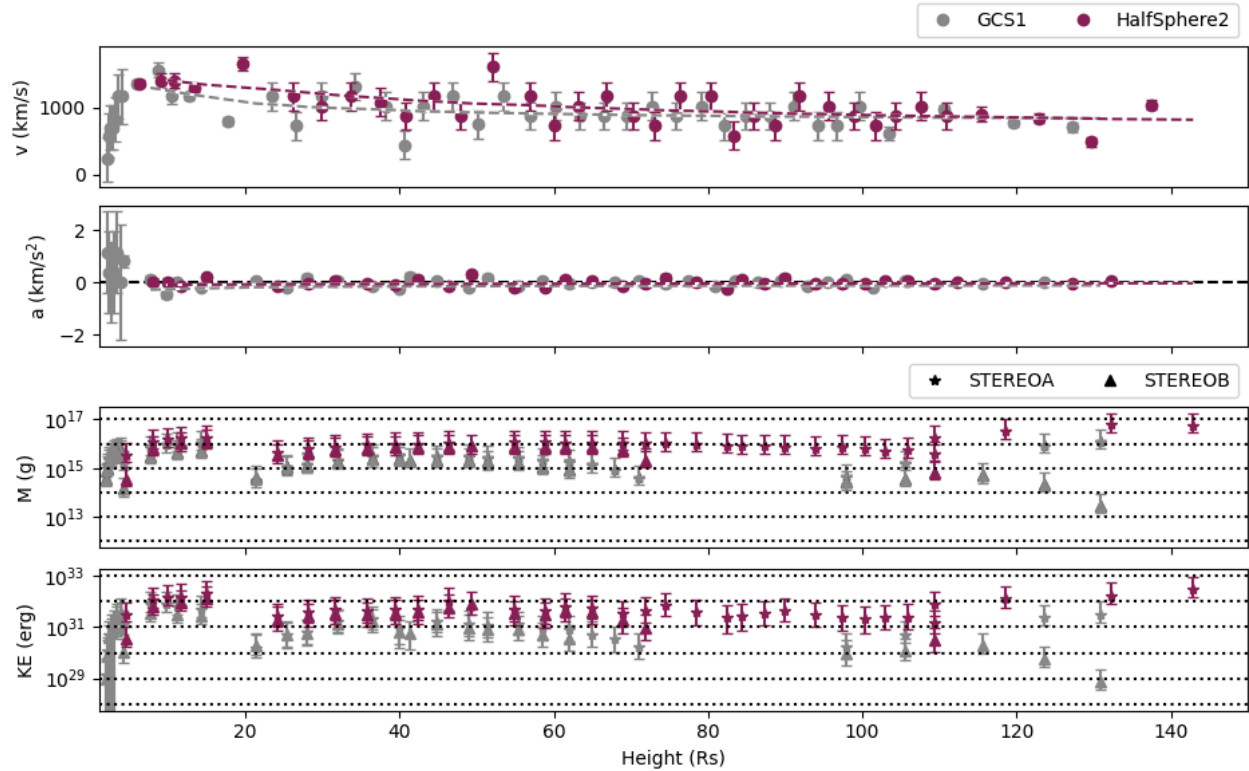
The WOMBAT package includes two functions, `wombatPlots.py` and `wombatMovie.py`, that use the logged results to make either figures or a movie. The plot script is called from the command line and functions similarly to using the main WOMBAT wrapper with a log file.

```
python wombatPlots.py logFile ids mode bonusParameters
```

where `logFile` is the path/name of the log file, `ids` is a string of line ids in the same format that WOMBAT uses, `mode` sets the plot type, followed by additional optional parameters. The primary types of figures are all line plots showing the evolution of various parameters versus time or height. There are three main modes - **ht#**, **kin#**, and **en#**, which refer to just a basic height profile, calculating kinematics, or calculating full energetics. The # can be replaced by an integer to indicate how many of the WF parameters to include. 1 indicates just the height, 2 is height and angular width(s), and 3 is all WF parameters. Below we show examples for `ht3` and `en1` plots.

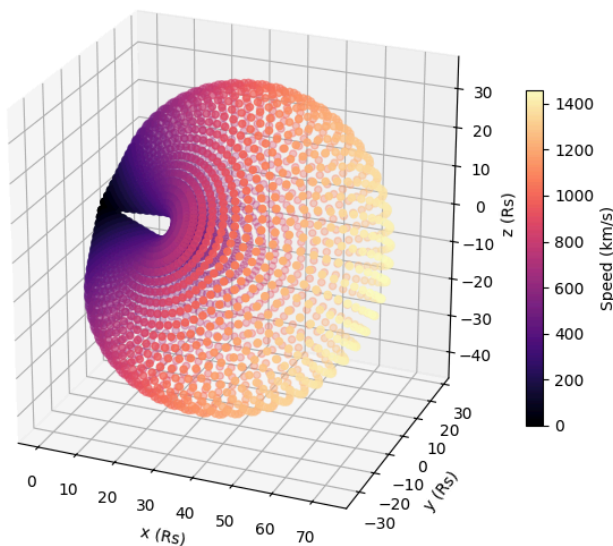
The kinematics are calculated using the height/time information. Basic two point derivatives are used to get the velocity, then derivatives of these used to calculate the acceleration. The code will check for multiple instances of the same reconstruction at slightly different times (due to the stagger in observation times) and only use a single average time for each height. The same velocity/acceleration is then assigned to each of the duplicate points. The energetics use DINGO (details coming below) to calculate the mass of the WF at each time for each instrument. The mass





is combined with the velocity to get kinetic energies.

The next mode is **pts** that will save the Cartesian Stonyhurst location of each point within a wireframe (defaults to `wombatPoints.txt` in `wbOutputs`). Each line of this file will have the WF type, time, point number, followed by x, y, and z in solar radii. Multiple log lines can be passed when using `pts`.



The final mode is **vmap** that will produce a 3D scatter plot showing the velocity of each point in a wireframe surface. To do this, two log lines for the same WF at different times must be given. The code will calculate the distance each grid point travels between the two times and show a scatter point at the average position colored by the corresponding velocity. Unless a save name is given as an optional parameter this will pop up an interactive python window where one can adjust the viewing angle. Note that the code does not require adjacent time steps to be used. It will happily calculate values using any time separation, though closer times (i.e. not

comparing EUV and HI) is typically recommended.

With the exception of `saveName`, the optional parameters configure the profile plots, not the other modes. Unless otherwise specified, these parameters are included simply by adding the given tag after the mode in the command line call. The available options are

- **eb** - flag to include error bars. These are calculated using general uncertainties for each reconstruction parameter and error propagation for the kinematics/energetics
- **drag** - flag to fit the standard drag equation to the reconstructions. The best fit for the initial CME speed, SW speed, height at which the fit starts, and drag coefficient will be printed to the screen and dashed lines will be included in the profile picture
- **vsh** - flag to plot profiles versus height instead of versus time
- **log** - flag to plot x-axis heights on a log scale
- **wfcolors** - flag to use the same colors as the GUI instead of the standard plot colors that were picked to work better for line plots on a white background. If you don't like these defaults you can also search for `colors2change` and set your own
- **png/pdf** - (pick one of the two) flag to set the save type to png or pdf (defaults to png)
- **1au/L1** - (pick one) flag to get a predicted arrival time at either 1 AU or L1. If drag has been flagged it will use the results of the drag fit, otherwise it just uses the final velocity and height. It will be ignored if not doing a kin or en plot.
- **densratio_#** - the ratio between the inner and outer wireframe ($\# = n_{In}/n_{Out}$). Or between the first and second if it cannot establish an inner/outer relationship based on type. The value is set by changing the $\#$ in the tag to an integer or float. This is only used if calculating masses for energetics
- **dh1_#/dh2_#** - the min/max heights to consider when trying to find the optimal starting height for the drag calculation (defaults to 5 Rs/21.5 Rs). The value is set by changing the $\#$ in the tag to an integer or float. This is only used if drag is flagged
- **newbase_#** - a flag to redo the mass calculation using a different base time than whatever was provided when the WOMBAT background pickle was created. This is done by setting $\#$ to a date, preferably of the form YYYY-MM-DDTHH:MM but really any string `parse_time` will accept. It does not have to be an exact time in the observations, the code will pick the closest match. This is only used if calculating masses for energetics
- **pickleName** - a string that contains `.pkl` in it. The mass calculation can be slow and one may want to revise plot aesthetics without waiting for it so the results are saved in a pickle in `wbPlotPickles`. If no name is provided then the calculation will run and store the results in `bigMassRes.pkl`. If a string is provided it will check if that exists and load it instead of calculating. If it does not exist it runs the calculation and saves it using that name. This is only used if calculating masses for energetics
- **saveName** - a name to save the figure (or points data). If the figure isn't given a name it will pop up in a window instead. It does not need the file extension.

Some of these tags might have similar options that also work (e.g. `versush`, `errorbars`) but they're not being listed here to avoid information overload. The details are in `processBonusArgs` if one has strong feelings about their favorite tags.

The other post-processing script makes movies using the results stored in a log file. This runs through `wombatMovie.py`, which is the only WOMBAT that is not simply configured in the command line at run time. There are a significant number of options to customize so the user needs to open the python file and edit the settings at the top of the code. These are all set and thoroughly described in the top ~100 lines of code, nothing should be changed below the “No touching!!!” block in standard use. These lines set the log file path, output movie name, an id string that functions the same as for the GUI/plots, the time resolution, number of columns in the movie grid, whether or not to include clean images (without WF projections), the frames per second, allow setting a custom order of the instruments, sets the background difference and scaling modes, WF points size, flag to include the overview window, options for custom colors, showing a smaller cutout of a window, and setting the background contour min/max values.

One all the parameters are specified simply run

```
python wombatMovie.py
```

and it will create the desired movie in the main wombat directory. This script takes a little time to run but nothing substantial, likely of order a minute if many instruments and time steps are included. If you get a codec error see the note about ffmpeg installation in Appendix 1.

7. DINGO

DINGO is the companion code to WOMBAT that determines densities within the 3D WF structure. The mass image (calculated from the total brightness during standard WOMBAT prep) provides a total mass per pixel. DINGO calculates the width of the WF perpendicular to the PoS for each pixel and calculates the density using the pixel area, perpendicular WF width, and the assumption of constant density along each LoS. Note that the WOMBAT GUI treats each FoV as a flat plane determined by the Thomson distance at the center of the FoV, but DINGO does account for the warping effects near the edges of the wide field observations. DINGO also accounts for the WF location relative to the PoS, using the average distance of the WF from the PoS at each pixel to adjust the projected mass according to the Billings equation.

7.1 Running DINGO

Before running DINGO, you need to have logged WOMBAT reconstructions within a log file. Identify the line numbers of the reconstructions of interest and call

```
python dingo.py logFile ids dim bonusParameters
```

where `logFile` is the path/name of the log file, `ids` is an index string in the same format used by WOMBAT, and `dim` is the desired dimension of the results (od - 3d). These first three arguments must be provided in order

DINGO has the ability to separate two different types of WFs. It first checks if one represents an outer sheath-like region (sphere and ellipse and their half versions) and one represents an inner CME-like structure (GCS, Torus, Tube, GCS*). In this case the inner is assumed to fully occupy its defined volume and the outer volume will have a cavity at the inner

region. If the types can not be nested as inner/outer then they each occupy their expected volume and both are allowed to exist in the overlapping region. While this is physically unrealistic, it is computationally necessary without additional information on how to split them. At this time DINGO can only separate 2 wireframes.

DINGO can process a time series of reconstructions. If using two WF shapes it will pair up entries with the same time and skip any entry without a pair. If running 2D mode then there is a maximum of 8 times for plotting aesthetics. For 3D only a single time may be used.

The dimension tag sets which form of output DINGO will return. Only one type can be used at a time. The options are

- **0D** - the total mass per WF
- **1D** - a set of profiles showing the in place and in situ densities
- **2D** - contours of the density
- **3D** - an interactive scatter plot of a density cloud (3D)

The dimension tag should be followed by any bonus parameters. Many of these parameters are identified by using a prefix followed by an underscore and a number. For these, the number at the end sets the value of prefix (e.g. prefix_0.5 would tell DINGO to set prefix at 0.5). Nothing is case sensitive. The optional parameters are:

- **Target** - the name of the in situ satellite of interest. This is required for 1D mode. Currently supports ACE, BepiColombo, DSCOVR, MAVEN, PSP, SolO, STEREO-A, STEREO-B, VEX, and Wind
- **Save name** - a string name tag to add to the figures/outputs. If none is provided then the figures are shown in a pop up window and not saved. DINGO automatically determines a bonus parameter is the save name if it doesn't fit any of the other categories. You cannot use one of the target satellite names as the save name
- **expf1_#** - the nonradial expansion factor calculated as the ratio at the time of impact compared to what it would be for self-similar expansion. The number should be a float greater than zero but not much more than one. *Only used in 1D mode*
- **expf2_#** - the radial expansion factor calculated as the radial expansion speed divided by the radial propagation speed. The number should be a decimal value, likely closer to zero than one. *Only used in 1D mode*
- **densratio_#** - the ratio of the density in WF1 to WF2, assumed to be constant across all overlapping pixels. This is only required when two WF types are included and will default to 1 (equal densities) if not provided.
- **vcme_#** - the average radial propagation speed of the CME in km/s. The number should be in the hundreds and will default to 400 if not specified. *Only used in 1D mode*
- **ds_#** - an integer indicating how much to downsample the resolution of the mass image when generating results. We default to full resolution for all modes except for 3D, for which we use 8 to avoid overloading the interactive scatter plot. This may need to be adjusted depending on the height/type of observation
- **newbase_#** - an option to recalculate the masses using a new base time, set by replacing # with a time in the form YYYY-MM-DDTHH:MM (or anything that

parse_time likes). The code will pick whatever is the closest available time to the input value, it does not have to be an exact match

- **logPlot** - a flag to show 2D contours on a log scale instead of linear. Defaults to False
- **projOff** - a flag to not correct the masses for the PoS separation before determining densities. Defaults to False (deprojection step included)
- **picType** - save images as png or pdf (defaults to png)
- **min_#/max_#** - set the minimum and maximum values of the contour levels. The number can be in any format accepted by the float command (e.g. 2.3 or 2e-12)
- **figsize_#x#** - the size of the figure (width x height) in inches. The numbers can be decimals and the two values must be separated by an x

7.2 DINGO Results

When DINGO is run it will first perform the mass to density calculation, separating multiple WFs if needed. This step can be a little slow but it will display the progress as it processes each frame. At this time it may warn about large deprojection factors. The Billings formula increases rapidly at large PoS separations so we cap it at a factor of 10, which is around 80° separation. Sometimes this occurs for a small number of points when looking at WFs close to the Sun as a small physical distance can create a large angular separation. The warning indicates the number of points and percentage of the total number that have been capped. If the number is small the results should be fine but if the percentage is large then determining masses for the event is not wise and the results should not be trusted.

oD Mode

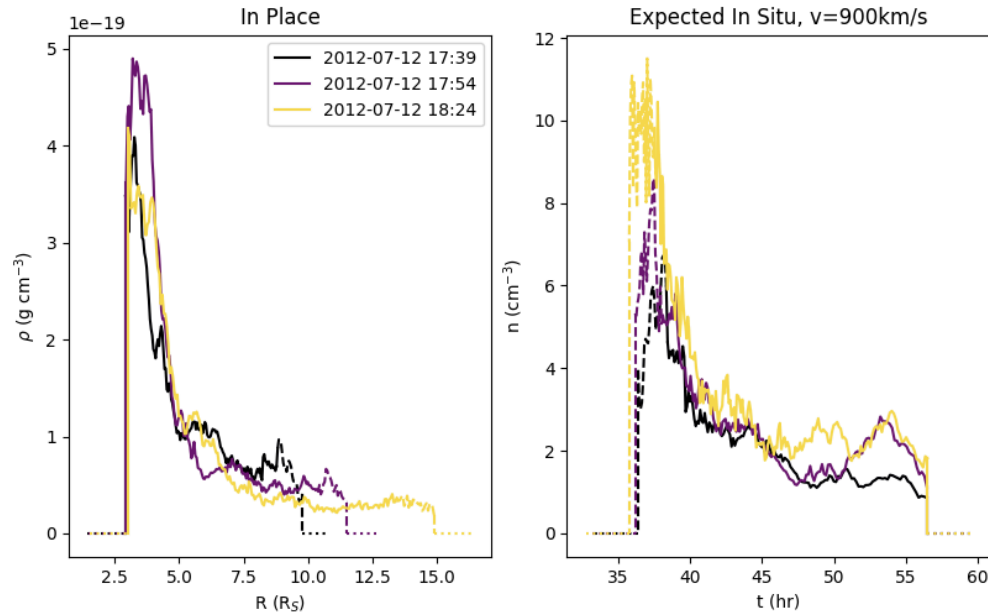
Running oD will produce a series of lines to the terminal displaying the total mass for each WF. For example, using two times and two WFs would result in

```
Time Inst WF1 WF2 Mass1(1e15 g) Mass2(1e15 g)
2012-07-12T17:39:00 COR2A GCS HalfSphere 7.97 14.25
2012-07-12T18:24:00 COR2A GCS HalfSphere 10.91 20.07
```

where the first line is a header then the results are listed below.

1D Mode

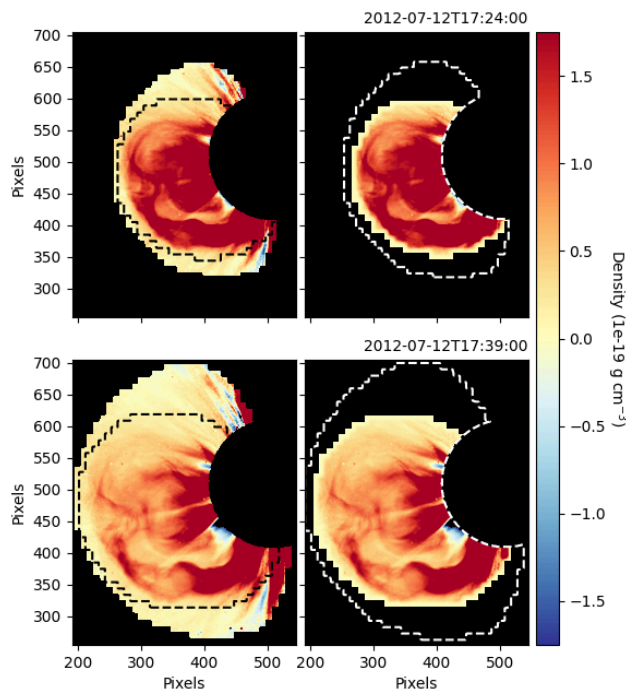
Running 1D will create a pair of line plots as shown below. DINGO determines the radial path connecting the Sun center to the satellite of interest. The left panel shows the “in place” density, which is the density along this radial path as observed by the remote satellite. The right panel shows in situ densities where the in place densities have been propagated to the target satellite distances. This uses an extremely simple propagation model using a constant radial propagation speed, a constant ratio of radial expansion to radial propagation (expf2), and a constant ratio of the perpendicular size upon arrival compared to the value expected from self similar expansion (expf1). ***This is not a proper arrival time model and should not be used as such***, it simply allows us to show the in situ results in terms of time instead of distance. The expansion factors cause a change in volume, which changes the density from the in place values, so the results are quite sensitive to their chosen values.



Both panels combine the results from all the reconstructions with different colors indicating different times according to the legend. The dotted portion of each profile represents outside any WFs, the dashed part represents within

the outer WF, and the solid line within the inner WF. Note that the left panel shows the in place density in g cm^{-3} whereas the right panel shows the in situ number density in cm^{-3} .

If a save name is provided then 1D DINGO will save these results in two text files (dingo1d_IP_saveName.dat and dingo1d_IS_saveName.dat). The IP file has the columns with the time of the remote reconstruction, the radial distance, the in place mass density, and a flag for whether that distance corresponds to the inner or outer WF. The IS file has the time of the remote reconstruction, the time elapsed since the earliest remote observation, the number density, and the inner/outer flag.



2D Mode

The 2D creates contour plots for each remote observation. The example below has two times and two WFs. The left panel shows the outer WF and the right panel shows the inner WF. In the left panel the dashed black line shows the boundary of the inner WF and conversely the dashed white line shows the boundary of the outer WF in the right panel.

The times are shown from top to bottom in chronological order. If more than four times are selected then it will split the times into two columns, keeping outer/inner pairs together if needed. The same color bar and plot range are used in all panels.

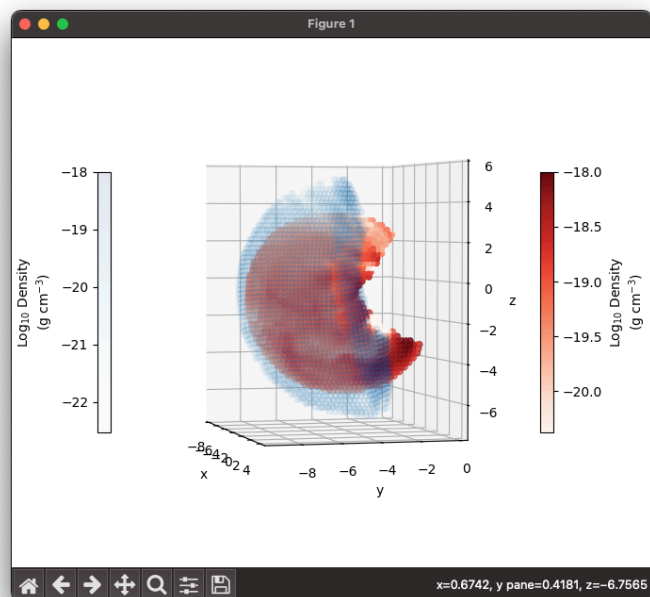
The code is set to try to create an aesthetically nice figure for all combinations of one or two WFs, 1-8 time steps, and different field of view shapes. This will likely not always work but one can manually adjust the figure size using the figsize tag. The figure size can also be adjusted by changing the python pop up window size. If you need to further adjust the whitespace padding you search for the rval/lval variables in the code.

If you give DINGO a single time step and instrument you can change the 2d tag to 23D to see the 2D plot mapped onto the PoS in 3D (not shown in this guide). This is relatively pointless for the coronagraphs but may add context for the wide field imagers. 23 dimensions is too many, its just 2 in 3D

3D Mode

The 3D DINGO option creates an interactive window where one can rotate the view as desired. The coordinates are “FoV Cartesian” where the flattened PoS lies within the yz plane at $x = 0$ and the remote satellite is on the x-axis.

Each scatter point is colored according to its density and when two WFs are used the outer is set to be semi-transparent. This makes it difficult to really infer much about relative densities, so the 2D contours are recommended in this situation, but there is some merit in being able to examine the expanded structure. The figure window can be resized to optimize the amount of white space around the grid depending on the point of view. The built in save function can be used to save images (or just screen cap). The min/max keywords also work for these contour levels.



8. Example Cases

We suggest two example cases to use to familiarize oneself with the WOMBAT GUI and pull/process procedures. Both cases were observed by a large number of instruments from different perspectives and therefore optimal cases for WF alignment. The table below provides a suggested time range, instruments, and time resolution to use for WOMBAT. The first case is the 2012 July 12 CME, which was a large event directed near Earth when both STEREO were still functional. This event has been extensively covered in the literature. The second case is 2023 September 24 when a series of eruptions occurred that were well-observed remotely by both SolO and PSP. [Liberatore+ 2025](#) provides a nice overview of this event.

Case	Inst Tags	Time Range/Res
1	AIA171, AIA 193, AIA 304, EUVI171, EUVI195, EUVI304 COR1, COR2, C2, C3 HI1, H2	2012/07/12 14:00-20:00 (10 min) 2012/07/12 14:00 - 2012/07/13 05:00 (20) 2012/07/12 14:00 - 2012/07/13 10:00 (20)
2	AIA171, AIA 193, AIA 304, EUVI171, EUVI195, EUVI304 COR1, COR2, C2, C3 HI1, H2, WISPR, SolOHI	2023/09/24 08:00-22:00 (20 min) 2023/09/24 08:00-22:00 (20) 2023/09/24 08:00- 2023/09/25/10:00 (30)

We have grouped the instruments and time range/resolution split by type (EUV/COR/HI). All values are just suggestions, not specific numbers that must be adhered to. Using case 1 as example, we recommend setting up the observations/GUI as below

Pull Calls

```
python3 wombatPullObs.py 2012/07/12T14:00 2012/07/12T20:00 AIA171 EUVT10
python3 wombatPullObs.py 2012/07/12T14:00 2012/07/12T20:00 EUVI171 COR1 COR2 C2 C3
HI1 HI2 EUVT10 CORT20 HIT20
```

Prep Call

Download COR1 files from [NASA site](#) for 2012/07 (both A and B)

```
python3 wombatProcessObs.py 2012/07/12T14:00 2012/07/13T10:00 AIA171 EUVI171 COR1
COR2 C2 C3 HI1 HI2
```

GUI Call

```
python3 wombatWrapper.py wbPickles/WBGUI_temp.pkl
```

We recommend breaking up the AIA pull calls as they are the most finicky and may potentially fail. The remaining instruments can be grouped together. The prep code should be fine processing all instruments at once, but keep in mind that WOMBAT will open one panel for each instrument and the screen will quickly become cluttered.

Appendix

A1. Python Packages and Versions

WOMBAT was developed using Python 3.12.4 with packages installed via pip. Below is a list of packages and their versions used during the development. These (and their dependencies) are included in requirements.txt, which can be loaded using

```
pip install -r requirements.txt
```

We also provide wombat.yml, which one can use to initiate a conda environment. We recommend following [these instructions](#) for setting up miniconda. The wombat environment can then be created as

```
conda env create -f wombat.yml --name wombat
```

and initiated via

```
conda activate wombat
```

and deactivated after use by

```
conda deactivate
```

Note that if using a conda environment spiceypy will print an INFO warning about a fallback library but proceed without issue.

Required packages:

aiapy 0.10.2, astropy 7.1.1, numpy 2.3.4, pyqt5 5.15.11, pyqtgraph 0.13.7, scipy 1.16.3, spiceypy 8.0.0, sunpy 7.0.3, wget 3.2

Additionally we import datetime, itertools, math, os, platform, and sys, but these should all be part of the main Python distribution.

Note on FFMPEG - saving the movies from the figure window animations requires some form of package to actually write the movie. WOMBAT uses ffmpeg as it was the recommended method and a well supported package. The imports don't explicitly call it but it is needed. For some unknown reason (path issues?), Python will return a codec error if you have installed FFMPEG via pip. If you get this error, uninstall via pip and reinstall via brew.

A2. Kernels

The first zip file from the WOMBAT Zenodo archive contains a set of SPICE kernels. The WOMBAT processing scripts depend on prep routines that use SPICE kernels for satellite pointing information. If you have IDL or the Python SunPy/spiceypy library installed then you may already have the needed files. We include a folder with all the necessary kernels in the WOMBAT repository and the code is set up to point to this folder, so a user should not need to perform any additional actions. We note these kernels may need to be updated over time and will attempt to keep the repository up to date. We list the kernels that we include in the package but we will not go into further detail on spice kernels. We recommend consulting the [NAIF](#) or [spiceypy](#) site for more information on kernels. All kernels in the repo have been pulled from

solarsoft but are also available online via either the NAIF site or individual satellite sites (theoretically).

There is a basic set of four kernels that most instruments will make use of, then a few larger satellite-specific sets. We include the necessary kernels in the repository but it may be necessary to update these in the future.

Basic kernels

The basic kernels are:

naif0012.tls, heliospheric.tf, pck00011_n0066.tpc, De421.bsp

Satellite specific kernels

As of WOMBAT 1.0, PSP, Solar Orbiter, and STEREO require satellite specific kernels. Each satellite has its own folder within the spiceKernels folder. The subfolders for each satellite separate between general, attitude, orbit, and sclk and follow the naming convention and structure as used by solarsoft. We note that the solarsoft versions contain more kernels than the WOMBAT set as we have only included the necessary ones.

Appendix 3. Prep Files

The second zip file from Zenodo contains most of the files necessary to process the background images. The only missing files are the COR1 pB files that must be downloaded manually for a specific month.

PSP WISPR

The prep code depends on 4 .sav files (created by IDL) and 3 fits files

- rollcomp.sav
- rollcompm180.sav
- rollcompp180.sav
- wcs0.sav
- WISPR_FredVignettingFitted_inner_20180809_01.fits
- WISPR_FredVignettingFitted_outer_20180809_01.fits
- WISPR_FredVignettingFitted_outer_20190813_01.fits

SOHO LASCO

The prep code depends on a large-ish folder of .dat files for the expfac and 6 fits files. The expfac_data folder contains a folder for each month (Jan 2001 - July 2024, so may need to be updated) that contains the appropriate file for each day. The fits files are

- 3m_clcl_all.fts
- 2vig_final.fts
- c3_cl_mask_lv1.fts
- C3ramp.fts
- c3vig_postint_final.fts
- c3vig_preint_final.fts

STEREO EUVI

The prep code depends on `secchi_gtdbase.geny` to process EUVI images and a series of wavelength specific fits files in `stereo/calimg`

STEREO HI

The prep code depends on a reasonably large folder of fits files accessed by `hi_fix_pointing` within the `hi_prep` code, as well as some fits images in `stereo/calimg`

Appendix 4. Other Fits Sources

The following sites can be used to directly download the data that WOMBAT needs.

[LASCO data at NRL](#) - This link corresponds to a specific day because if one tries to go to the directory one above it is quite laggy trying to load the list of all available dates. Simply replace the '260101' in the address with the YYMMDD of your choice. These file names are not dated so Dump everything in `pullFolder/SOHO/LASCO/notDated/` and make sure you have the `img_hdr.txt` file for that day (at the bottom of the webpage). Run the `renameLASCO` helper function (in `processCode`) and it should move the numbered files into either the C2/C3 folder and match their names to the WOMBAT pulled versions. Note that `img_hdr` file may throw an error from the `genfromtxt` command having different number of columns. This occurs when some of the column warning flags use a phrase with spaces (e.g. -60 De or Seq PW). Either make them spaceless (e.g. SeqPW) or just delete that whole row and remove the corresponding fits files. We don't want these files anyway for the various reasons they have been flagged.

[SoloHI data at NRL](#) - The same data as from the pull script. Results do need to be separated by camera 1-4 in the `pullFolder`

[STEREO SECCHI data at APL](#) - The link has all the data, pick either STA or STB, then the `img` data, then your instrument of choice. This site is pretty slow to load once you select an instrument because it loads a folder for each date. The Fido pull routines tend to be pretty reliable for STEREO so `wombatPull` is the preferred method.

[WISPR data at NRL](#) - This site has the L2, L3, and LW files. The folder structure should be intuitive and the file names will match the WOMBAT pulled versions

TBD on EUV... not the primary focus of wombat

Appendix 5. Time Complications

A complicated note on time that explains some peculiarities - WOMBAT contains a time series for both the global GUI time (as given by the time slider) and for each instrument, each with their own cadence. The global time is usually the highest resolution so everything defaults/refers to its indexing of time. At startup, WOMBAT creates a map from global time index to each instrument time index by determining which observation time is closest at each global time. This map is different for each instrument. The "current" time is defined as the range of global time over which there is no change in any of the matching observation times.

For example, consider a set up where the global time has six times/indices (GIdx) or [0,1,2,3,4,5]. Over this same time range we have two instruments, one with three observations and the other with two. These instruments have maps from GIdx to instrument indices (IIdx) as [0,0,1,1,2,2] and [0,0,0,1,1,1] (where IIdx = map[GIdx]). If the global time is set at GIdx 1 then instrument 1 is at IIdx 0 and instrument 2 is at IIdx 0. The GUI would determine the “current time” as GIdx [0,1] because that is the range where both instruments keep the same IIdx that they currently have. If the GIdx is moved from 1 to 2 then the observation settings would be saved for GIdx [0,1] (IIdx [0]) for instrument 1 and GIdx [0,1,2] (IIdx [0]) for instrument 2. The WF settings would be saved for GIdx [0,1] because that is the current time.

This is a simple example but likely still hard to follow a text description. In summary, there are multiple timelines with different resolutions. WOMBAT saves things at matching times, but matching gets complicated. If something looks off in an observation window, make sure that the observation time is close to the global time, it may be a global time step or two away from triggering the switch in parameters if these times are misaligned.

Ongoing Work

Known “Issues”

- Pull script was weird about pulling more than 1 day at a time but then issue disappeared. Quantum Fido issue maybe?

To Dos:

- Instruments to add:
 - ASPIICS, CCOR, HMI, METIS, PROBA, PUNCH, SWAP
- Couple with OSPREI?

WOMBAT Cheat Sheet

Basic Steps

1. Pull Data python3 wombatPullObs.py <i>t1 t2 insts tRes</i>	2. Process Data python3 wombatProcessObs.py <i>t1 t2 insts pickle</i>
3. Run WOMBAT python3 wombatWrapper.py <i>pickle nWFs ovw</i> -or- python3 wombatWrapper.py <i>logFile logIDs ovw</i>	4. Run DINGO python3 dingo.py <i>logFile logIDs dim optParams</i>

Arguments

Times (t1/t2): start and stop times as YYYY/MM/DDTHH: MM (but really anything parse_time will accept)	Instruments (insts): EUV - AIA#, EUVI#A, EUVI#B COR - C2, C3, COR1A***, COR1B***, COR2A, COR2B HI - HI1A, HI1B, HI2A, HI2B, SoloHI, SoloHI1-4, WISPR, WISPRI, WISPRO	EUV Wavelengths: EUV tags need to include wavelength AIA: 94, 131, 171, 193, 211, 304, 335, 1600, 1700 EUVI: 171, 195, 284, 304, EUI: 174, 304
Time res (tRes): EUVI#/CORT#/HIT# (e.g. CORT20) Defaults to 10/20/30 min	Files: <i>pickle</i> - processData creates (can set name to avoid overwriting) <i>reloadFile</i> - from wombat save <i>logFile</i> - from wombat log <i>dingo_config</i> - user-generated	WOMBAT Other Params: nWFs: # of wireframes (max 5, default 1) ovw: flag to turn on overview window (defaults off)
DINGO Dimensions: 0 - Total mass 1 - Density profiles 2 - Density contours 3 - Interactive scatter plot	logIDs - id1+id2+... Line numbers in logFile for the recons of interest separated by + (e.g. '1+2' means 1 and 2)	DINGO Optional Params: Req for 1D: target Prefix+value: expf1_#, expf2_#, densratio_#, vcme_#, ds_#, min_#, max_#, figsize_#x# Flags: projoff, logplot, Others: pictype, saveName

*** Download any COR1 background files from [NASA site](#) before running processing

WOMBAT Key Shortcuts

Q	Quit window	(⇧)M	Mass calculation	(⇧)L	Log recon
ESC	Quit everything	(⇧)R	Running diff mode	(⇧)S	Save figure
H	Show/hide WFs	(⇧)B	Base diff mode	⇧7	Copy WF params ←
←	Step to earlier times	(⇧)1	Linear scaling	⇧8	Copy WF params →
→	Step to later times	(⇧)2	Log scaling	⇧9	Copy fig min/max ←
		(⇧)3	Square root scaling	⇧0	Copy fig min/max →

(⇧) indicates that pressing that just the key will perform the action for the current time but shift+key will do it for all times. The entries without () around ⇧ only operate when shift is pressed. These actions work on a single obs window if it is selected or on all the obs windows when the param is window is active